

**1. Build a basic server using Node.js and Express. Create routes to handle HTTP requests like GET and POST and respond with simple JSON data.**

```
const express=require("express") const app =
express();

const PORT = process.env.PORT || 3000;

app.use(express.json());
app.use(express.static("public"));

const data=[
    {id:1, name:"Abhishek"},
    {id:2, name:"Bhuvan"},
    {id:3, name:"Chetan"},
    {id:4, name:"Deksha"},
];

app.get("/",(req,res)=>{ res.send("Home");
});

app.get("/api/data",(req,res)=>{ res.json(data);
});

app.post('/api/data',(req,res)=>{ const
    nItem=req.body; data.push(nItem);
    res.json(data);
});

app.listen(PORT,(err)=>{
    if(err)console.log(err);
    console.log("Server running on port:",PORT);
});
```

**2. Extend the previous Node.js server by integrating a database (e.g., SQLite or MongoDB). Implement endpoints to perform CRUD operations on a dataset.**

```
const express = require("express"); const mongoose
= require("mongoose");

const app = express(); const
PORT = 3000;

app.use(express.json());

// MongoDB connection mongoose.connect("mongodb://127.0.0.1:27017/studentDb")
  .then(() => console.log("MongoDB connected!"));

// Schema
const studentSchema = new mongoose.Schema({ id:
  Number,
  name: String,
});

// Model
const Student = mongoose.model("Student", studentSchema);

// GET all
app.get("/data", async (req, res) => { const data =
  await Student.find(); res.json(data);
});

// GET by id
```

```
app.get("/data/:id", async (req, res) => {  
  const data = await Student.findOne({ id: Number(req.params.id) }); res.json(data);  
});
```

```
// POST
```

```
app.post("/data", async (req, res) => { const newData =  
  new Student(req.body); const saved = await  
  newData.save(); res.json(saved);  
});
```

```
// PUT
```

```
app.put("/data/:id", async (req, res) => {  
  const updated = await Student.findOneAndUpdate(  
    { id: Number(req.params.id) }, req.body,  
    { new: true }  
  );  
  res.json(updated);  
});
```

```
// DELETE
```

```
app.delete("/data/:id", async (req, res) => { const deleted = await  
  Student.findOneAndDelete({  
    id: Number(req.params.id),  
  });  
  res.json(deleted);  
});
```

```
// Start server app.listen(PORT, ()  
=> {
```

```
    console.log(`Server running on port ${PORT}`);  
  });
```

### **3. Design and implement a RESTful API using Node.js, Express, and a database of your choice. Define endpoints for managing resources, such as creating, reading, updating, and deleting data.**

```
const express = require("express"); const mongoose  
= require("mongoose");
```

```
const app = express(); const  
PORT = 3000;
```

```
app.use(express.json());
```

```
// MongoDB connection mongoose.connect("mongodb://127.0.0.1:27017/studentDb")  
  .then(() => console.log("MongoDB connected!"))  
  .catch(err => console.log(err));
```

```
// Schema  
const studentSchema = new mongoose.Schema({ id:  
  Number,  
  name: String,  
});
```

```
// Model  
const Student = mongoose.model("Student", studentSchema);
```

```
// Root  
app.get("/", (req, res) => { res.send("Student API is  
  running...");  
});
```

// CREATE

```
app.post("/data", async (req, res) => { try {  
    const newStudent = new Student(req.body); const  
    saved = await newStudent.save(); res.json(saved);  
  } catch (err) { res.status(500).send("Error");  
  }  
});
```

// READ ALL

```
app.get("/data", async (req, res) => { try {  
    const students = await Student.find();  
    res.json(students);  
  } catch (err) {  
    res.status(500).send("Error");  
  }  
});
```

// READ ONE

```
app.get("/data/:id", async (req, res) => { try {  
    const student = await Student.findOne({ id:  
    Number(req.params.id) });  
    res.json(student);  
  } catch (err) { res.status(500).send("Error");  
  }  
});
```

// UPDATE

```
app.put("/data/:id", async (req, res) => { try {  
    const updated = await Student.findOneAndUpdate(  

```

```
        { id: Number(req.params.id) }, req.body,
        { new: true }
    );

    res.json(updated);
  } catch (err) { res.status(500).send("Error");
  }
});

// DELETE
app.delete("/data/:id", async (req, res) => { try {
    const deleted = await Student.findOneAndDelete({ id:
        Number(req.params.id),
    });
    res.json(deleted);
  } catch (err) { res.status(500).send("Error");
  }
});

// Start server app.listen(PORT, ()
=> {
    console.log(`Server running on port ${PORT}`);
});
```

#### **4. Create a library of reusable React components. Build components like buttons, cards, and modals and use them in a sample React application.**

##### **App.js**

```
import React, {useState} from "react"; import
"./App.css";
```

```
import Button from "../components/Button"; import
Card from "../components/Card"; import Modal from
"./components/Modal";

function App(){
  const [showModal, setShowModal]=useState(false);
  return(
    <div className="app">
      <Button text="Open Modal" onClick={()=> setShowModal(true)}
    />
      <Card>
        <h3>React Card</h3>
        <p>This is a reusable card component.</p>
      </Card>
      <Modal show={showModal} onClose={()=>setShowModal(false)}>
        <h2>Modal Window</h2>
        <p>This is a reusable modal component.</p>
      </Modal>
    </div>
  );
}

export default App;
```

## Button.js

```
import React from "react";
```

```
function Button({text, onClick}){ return(  
    <button className="button" onClick={onClick}>  
        {text}  
    </button>  
    );  
}  
  
export default Button;
```

## Card.js

```
import React from "react";  
  
function Card({children}){  
    return(  
        <div className="card">  
            {children}  
        </div>  
    );  
}  
  
export default Card;
```

## Modal.js

```
import React from "react";  
  
function Modal({show, onClose, children}){ if(!show){  
    return null;  
}  
}
```



```
    return(  
      <div className="modal-overlay">  
        <div className="modal">  
          <button className="close-button" onClick={onClose}> X  
        </button>  
        {children}  
      </div>  
    </div>  
  );  
}  
  
export default Modal;
```

## App.cs

```
.app{  
  display:flex;  
  flex-direction:column; align-  
  items:center; justify-  
  content:center; height:100vh;  
}  
  
.button{ padding:10px  
  20px;  
  margin-bottom:20px;  
}  
  
.card{
```

```
border:1px solid #ccc;
padding: 20px;
margin-bottom: 20px; width:
300px;
text-align: center;
}

.modal-overlay{ position:
fixed; top: 0;
left: 0;
width: 100%;
height: 100%;
background: rgba(0,0,0,0.5); display: flex;
align-items: center; justify-
content: center;
}

.modal{
background: white;
padding: 20px; width:
400px; border-radius:
8px; position: relative;
}

.close-button{ position:
absolute; top: 10px;
right: 10px; cursor:
pointer; background:
none; border: none;
font-size: 18px;
```

```
}
```

## 5. Write a program to create a simple calculator Application using React JS.

*App.js*

```
import React, { useState } from 'react'; import
'./App.css';

const App = () => {
  const [question, setQuestion] = useState(""); const [answer,
  setAnswer] = useState("");

  const handleClick = (value) => { if (value
    === '=') {
      try {
        setAnswer(eval(question) || "Math Error"); setQuestion("");
      } catch {
        setAnswer("Math Error");
      }
    } else if (value === 'C') { setQuestion("");
      setAnswer("");
    } else if (value === 'CE') {
      setQuestion(question.slice(0, -1));
    } else {
      setQuestion(question + value);
    }
  };

  const buttons = ['1','2','3','4','5','6','7','8','9','-
    ','+', '0', '=', '*', 'C', 'CE', '/'];
```

```
    return (  
      <div className="calculator">  
        <h2>React Calculator</h2>  
  
        <div className="screen">  
          <input type="text" value={question} readOnly />  
          <input type="text" value={answer} readOnly />  
        </div>  
  
        <div className="buttons">  
          {buttons.map((btn) => (  
            <button key={btn} onClick={() => handleClick(btn)}>  
              {btn}  
            </button>  
          ))}  
        </div>  
  
      </div>  
    );  
  }  
};
```

```
export default App;
```

## App.css

```
body {  
  display: flex;  
  justify-content: center; align-  
  items: center; height: 100vh;
```

```
background-color: #f1f5f9; /* soft light gray */ font-family:
  Arial, Helvetica, sans-serif;
}
```

```
/* Calculator Container */
```

```
.calculator {
  width:300px;
  background:#1e293b;          /* dark slate */
  padding: 30px;
  border-radius: 10px;
  box-shadow: 0 6px 12px rgba(0,0,0,0.2); text-align:
  center;
  color: #ffffff;
}
```

```
h2{
  margin-bottom: 10px;
}
```

```
/* Screen */
```

```
.screen { display:flex;
  flex-direction: column;
  margin-bottom: 10px;
}
```

```
.screen input{
  width:100%;
  padding:10px; font-
  size:1.2em; text-
```

```
align:right;
margin-bottom: 5px; border:1px
solid #cbd5f5; border-
radius:5px;
background:#f8fafc;          /* light display */
color:#111827;
}
```

```
/* Buttons Grid */
.buttons{
  display:grid;
  grid-template-columns: repeat(4,1fr); gap:5px;
}
```

```
/* Buttons */ button{
  padding:15px; font-
size:1.2em;
border:none;
background: #e2e8f0;          /* light gray */
cursor:pointer;
border-radius:5px; transition:background
0.2s;
}
```

```
/* Hover */ button:hover{
  background: #cbd5f5;
}
```

```
/* Click */ button:active{  
    background: #94a3b8;  
}
```

## 6. Create a Simple Login form using React JS.

```
import React, {useState} from "react";
```

```
import "./prog6.css"
```

```
function Login(){  
    const [ username,setUsername]=useState("");  
    const [ password, setPassword]=useState("");  
    const handleLogin=()=>>{  
        if(username==="admin" && password==="123"){  
            alert("Login successful");  
        }else{  
            alert("Invlaid credentials");  
        }  
    };  
};
```

```
return(  
    <div>  
        <h2>Login</h2>  
        <input type="text" placeholder="Username"  
onChange={(e)=>setUsername(e.target.value)}  
        />  
        <br/><br/>  
        <input type="password" placeholder="Password"  
onChange={(e)=>setPassword(e.target.value)}
```

```
    />

    <br/><br/>

    <button onClick={handleLogin}>Login</button>

  </div>

);

}
```

```
export default Login;
```

## **7. Build a real-time chat application using React for the front end and Node.js with Socket.IO for the back end. Enable users to send and receive messages in real-time.**

**Frontend: React**

**Backend: Node.js + Express + Socket.IO**

**Create a folder: server**

### **1. Backend (Node.js + Socket.IO)**

**Install dependencies**

**npm init -y**

**npm install express socket.io cors**

**server.js**

```
const express = require("express");
const http = require("http");
const cors = require("cors");
const { Server } = require("socket.io");
```

```
const app = express();
app.use(cors());
```

```
const server = http.createServer(app);
```

```
const io = new Server(server, {
  cors: {
    origin: "http://localhost:3000",
```



```
    methods: ["GET", "POST"]
  }
});

io.on("connection", (socket) => {
  console.log("User connected:", socket.id);

  socket.on("send_message", (data) => {
    io.emit("receive_message", data); // broadcast
  });

  socket.on("disconnect", () => {
    console.log("User disconnected:", socket.id);
  });
});

server.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

**node server.js**

## **2. Frontend (React)**

```
npx create-react-app chat-app
cd chat-app
npm install socket.io-client
```

### **App.js**

```
import React, { useEffect, useState } from "react";
import io from "socket.io-client";

const socket = io("http://localhost:5000");

function App() {
  const [username, setUsername] = useState("");
  const [message, setMessage] = useState("");
  const [chat, setChat] = useState([]);
  const [joined, setJoined] = useState(false);

  const joinChat = () => {
    if (username !== "") {
      setJoined(true);
    }
  };
};
```

```
const sendMessage = () => {
  if (message !== "") {
    socket.emit("send_message", {
      user: username,
      text: message
    });
    setMessage("");
  }
};

useEffect(() => {
  socket.on("receive_message", (data) => {
    setChat((prev) => [...prev, data]);
  });
}, []);

return (
  <div style={{ padding: "20px" }}>
    {!joined ? (
      <div>
        <h2>Enter Username</h2>
        <input
          type="text"
          placeholder="Your name"
          onChange={(e) => setUsername(e.target.value)}
        />
        <button onClick={joinChat}>Join Chat</button>
      </div>
    ) : (
      <>
        <h2>Welcome {username}</h2>

        <div style={{
          border: "1px solid black",
          height: "300px",
          padding: "10px",
          overflowY: "scroll"
        }}>
          {chat.map((msg, index) => (
            <p key={index}>
              <b>{msg.user}</b> {msg.text}
            </p>
          ))}
        </div>

        <input
          type="text"

```

```
        value={message}
        onChange={(e) => setMessage(e.target.value)}
      />
      <button onClick={sendMessage}>Send</button>
    </>
  )}
</div>
);
}
```

export default App;

npm start.

## 8. Develop a Django app that displays current date and time in server

Step 1: install Django

```
python -m pip install Django
```

step 2: create Django app

```
python -m django startproject datetime_project
```

step 3: cd datetime\_project

step 4: Create App

```
python manage.py startapp timeapp
```

step 5: Register App

Open settings.py and add your app:

datetime\_project/settings.py

```
INSTALLED_APPS = [
```

```
    'django.contrib.admin',
```

```
    'django.contrib.auth',
```

```
    'django.contrib.contenttypes',
```

```
    'django.contrib.sessions',
```

```
    'django.contrib.messages',
```

```
    'django.contrib.staticfiles',
```

```
    'timeapp',    // add this line
```

]

Step 6 : Create a View

```
timeapp/views.py

from django.http import HttpResponse

from datetime import datetime

def current_datetime(request):

    now = datetime.now()

    html = f'<h1>Current Date and Time (Server): {now}</h1>'

    return HttpResponse(html)
```

step 7: Configure URLs

```
Project-level (datetime_project/urls.py)

from django.contrib import admin

from django.urls import path

from timeapp import views

urlpatterns = [

    path('admin/', admin.site.urls),

    path("", views.current_datetime),

]
```

Step 8: Run Server

```
python manage.py runserver
```

step 9: View Output

<http://127.0.0.1:8000/>

**Current Date and Time (Server): 2026-04-22 11:42:00.264552**

## 9. Develop a simple Django app that displays an unordered list of fruits and ordered list of selected students for an event.

### Create Project

```
python -m django startproject list_project
cd list_project
```

### Create App

```
python manage.py startapp listapp
```

### Register App

```
list_project/settings.py
```

### Create View

#### listapp/views.py

```
from django.shortcuts import render

def display_lists(request):
    fruits = ['Apple', 'Banana', 'Mango', 'Orange']
    students = ['Rahul', 'Anita', 'Kiran', 'Sneha']

    return render(request, 'listapp/index.html', {
        'fruits': fruits,
        'students': students
    })
```

### Create Template Folder

Create this structure manually:

```
listapp/
  templates/
    listapp/
      index.html
```

### Create HTML Template

**templates/listapp/index.html**

```
<!DOCTYPE html>
<html>
<head>
  <title>Lists Example</title>
</head>
<body>

  <h2>Fruits (Unordered List)</h2>
  <ul>
    {% for fruit in fruits %}
      <li>{{ fruit }}</li>
    {% endfor %}
  </ul>

  <h2>Selected Students (Ordered List)</h2>
  <ol>
    {% for student in students %}
      <li>{{ student }}</li>
    {% endfor %}
  </ol>

</body>
</html>
```

**Configure URLs****list\_project/urls.py**

```
from django.contrib import admin
from django.urls import path
from listapp import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", views.display_lists), # Home page
]
```

**Run Server**

```
python manage.py runserver
```

## View Output

Open browser:

<http://127.0.0.1:8000/>

## Output

### Fruits (Unordered List)

- Apple
- Banana
- Mango
- Orange

### Selected Students (Ordered List)

1. Rahul
2. Anita
3. Kiran
4. Sneha

**10. Develop a Django app that performs student registration on a course. It should also display a list of students registered for any selected course. Create students and course as models with enrolment as many to many fields.**

Creates a **Django project** named myproject This is the main container (settings, URLs, config)

```
python -m django startproject myproject
cd myproject
```

Creates an **app** called myapp

App = module where your actual logic (models, views) is written

```
python manage.py startapp myapp
```

Registers your app in Django Without this, Django will **ignore your app**

settings.py:

```
INSTALLED_APPS = ['myapp']
```

Models (models.py)

```
from django.db import models //Imports Django's database tools
```

```
class Course(models.Model): //Creates a table called Course
```

```
    name = models.CharField(max_length=50) //Column name (text, max 50 characters)
```

```
    def __str__(self): //Defines how object is shown (admin, shell)
```

```
        return self.name //Instead of Course object, it shows actual name
```

```
class Student(models.Model): // Creates table Student
```

```
    name = models.CharField(max_length=50)
```

```
    courses = models.ManyToManyField(Course) //One student → many courses
```

*One course → many students Django automatically creates a **third table** (join table)*

```
    def __str__(self):
```

```
        return self.name
```

### **Apply Migration**

```
python manage.py makemigrations // Converts models into migration files (instructions)
```

```
python manage.py migrate // Applies those instructions to database Tables are created in DB
```

View (views.py)



```
from django.shortcuts import render    // Used to send data to HTML page
from .models import Student, Course    // Imports your database tables

def home(request):                    // Function for homepage request contains user input (GET/POST)
    courses = Course.objects.all()    // Fetches all courses from DB
    students = None                   // Default value (no students shown initially)

    if request.GET.get('course'):      // Checks if user selected a course
        course_id = request.GET.get('course')    // Gets selected course ID from URL
        students = Course.objects.get(id=course_id).student_set.all()

    // Finds selected course Retrieves all students in that course

    return render(request, 'home.html', {
        'courses': courses,
        'students': students
    }) // Sends data to template courses and students available in HTML
```

## URL Configuration

### myapp/urls.py

// When user visits / → call home function

```
from django.urls import path
from .views import home
```

```
urlpatterns = [
    path("", home),
]
```

### main urls.py

**Connects app URLs to project Without this → page won't load**

```
from django.urls import path, include
```

```
urlpatterns = [
    path("", include('myapp.urls')),
]
```

## Simple Template (home.html)

Create a templates folder and add:

<h2>Select Course</h2>

```
<form method="GET">
  <select name="course">
    {% for c in courses %}
      <option value="{{ c.id }}">{{ c.name }}</option>
    {% endfor %}
  </select>
  <button type="submit">Show Students</button>
</form>
```

```
{% if students %}
  <h3>Students:</h3>
  <ul>
    {% for s in students %}
      <li>{{ s.name }}</li>
    {% endfor %}
  </ul>
{% endif %}
```

### Run Server

python manage.py runserver

python manage.py createsuperuser

Creates admin login

Use /admin/ to:

- Add courses
- Add students
- Assign courses to students